TRIBHUVAN UNIVERSITY

Institute of Engineering, Purwanchal Campus

Department of Electronics and Computer Engineering

Dharan, Sunsari

ARTIFICIAL INTELLIGENCE [ENCT 305]

LAB REPORT: TO IMPLEMENT K-MEANS CLUSTERING IN A GIVEN DATASET.

Submitted By

Abhishek Niraula

Roll no.: PUR080BEI004

Faculty: BEI

Submitted To

Er. Sujan Karki

Instructor

Purwanchal Campus, Dharan

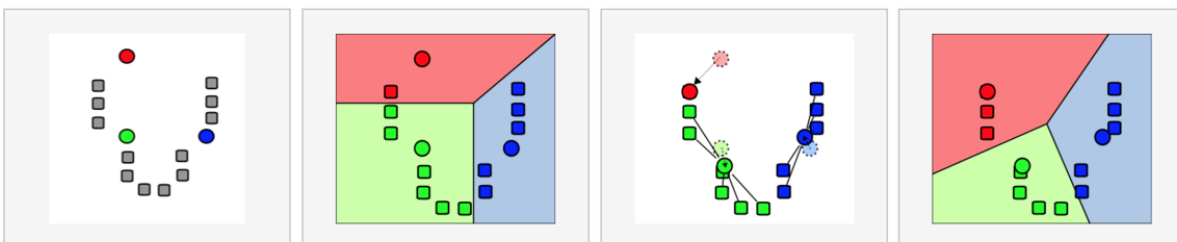
INTRODUCTION

K-means clustering is an unsupervised learning algorithm used for data clustering, which groups unlabeled data points into groups or clusters.

It is one of the most popular clustering methods used in machine learning. Unlike supervised learning, the training data that this algorithm uses is unlabeled, meaning that data points do not have a defined classification structure.

While various types of clustering algorithms exist, including exclusive, overlapping, hierarchical and probabilistic, the k-means clustering algorithm is an example of an exclusive or "hard" clustering method. This form of grouping stipulates that a data point can exist in just one cluster. This type of cluster analysis is commonly used in data science for market segmentation, document clustering, image segmentation and image compression. The k-means algorithm is a widely used method in cluster analysis because it is efficient, effective and simple.

Demonstration of the standard algorithm



1. *k* initial "means" (in this case *k*=3) are randomly generated within the data domain (shown in color).

2. *k* clusters are created by associating every observation with the nearest mean. The partitions here represent the Voronoi diagram generated by the means.

3. The centroid of each of the *k* clusters becomes the new mean.

4. Steps 2 and 3 are repeated until convergence has been reached.

K-means is an iterative, **centroid-based** clustering algorithm that partitions a dataset into similar groups based on the distance between their centroids. The centroid, or cluster center, is either the mean or median of all the points within the cluster depending on the characteristics of the data.

A) CLUSTER EVALUATION METRICS

Quality clusters contain at least two properties:

- All data points within a cluster should be similar.
- Clusters should be distinct from each other.

These properties are achieved by minimizing the intracluster distance and maximizing the intercluster distance of all data points in a dataset. In other words, the more compact and isolated a cluster is from other clusters, the better. The

goal of the k-means clustering algorithm is to minimize the **sum of squared errors** SSE^2**.** Computing the SSE of the squared **euclidean distance** of each point to its closest centroid evaluates the quality of the cluster assignments by measuring the total variation within each cluster.

Euclidean Distance?

Euclidean distance is the straight-line distance between two points in a Euclidean space (such as a 2D plane or 3D space), calculated using the Pythagorean theorem.

Cluster evaluation metrics check the quality and provide different perspectives to analyze clustering results. This helps in selecting the optimal number of clusters and centroid initialization. The following evaluation metrics are common ways to measure the intra and intercluster distances in clustering.

i) Inertia

The k-means clustering algorithm aims to choose centroids, or cluster centers, that minimize inertia, an evaluation metric that measures how well a dataset has been clustered based on distance metrics. Inertia is calculated by measuring the distance between a datapoint and its centroid, squaring the distance and summing those squares for each data point in the cluster. The sum or inertial value is the intracluster distance. The lower the sum the better because it means that the data points within the cluster are compact or more similar.

ii) The DUNN Index

The second property is measured with the Dunn index. The Dunn index represents the relationship between the minimum intercluster distance and the maximum intracluster distance. Clusters with a high intercluster distance indicate better quality because it means that the clusters are as different from each other as possible.4

B) OPTIMIZING K-MEANS PERFORMANCE

i) Initializing the cluster centroids

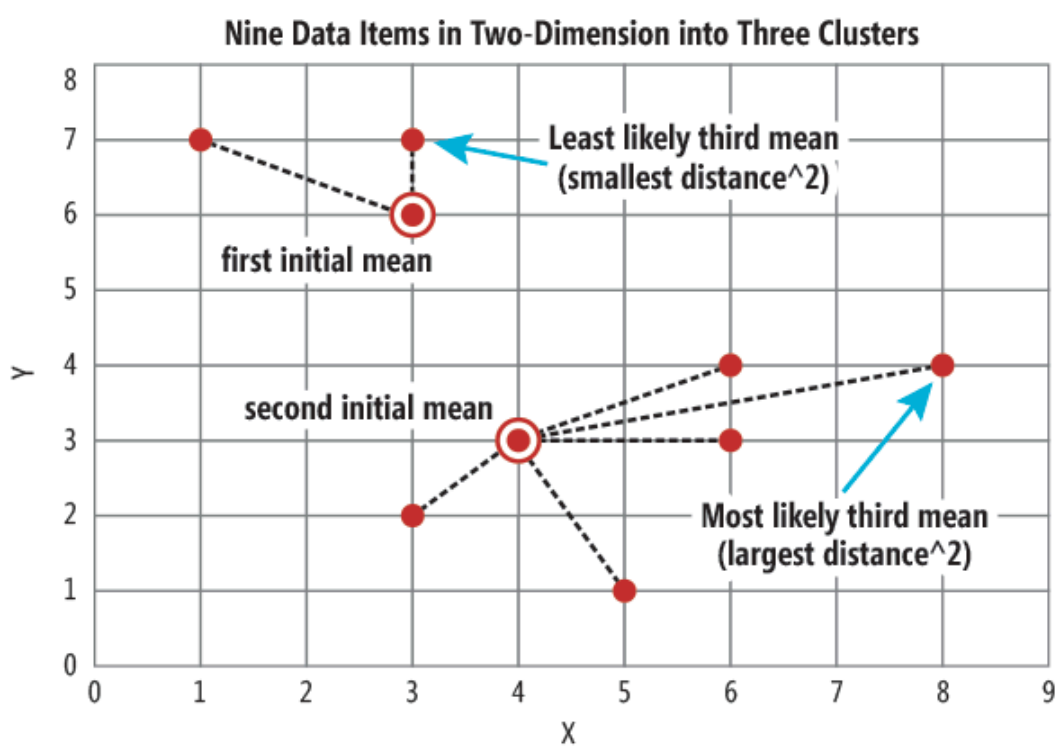
Each cluster is represented by a centroid, a data point that represents the cluster center. K-means groups together similar data points into clusters by

minimizing the distance between data points in a cluster with their centroid or k mean value. The primary goal of the k-means algorithm is to minimize the total distances between points and their assigned cluster centroid. The algorithm operates iteratively, and initial partition selection can greatly impact the resulting clusters.

Random initialization risks yielding inconsistent results. Centroid initialization methods exist to mitigate these risks. A study by **NUS Computing** explains and compares methods such as the popular k-means++ to random initialization.

K-Means++

The first step to initialization by using the k-means++ method is to choose one centroid from the data set. For each subsequent centroid, calculate the distance of each data point from its closest cluster center. The subsequent centroid is selected by considering the likelihood that a point is at a proportional distance from the nearest centroid chosen earlier. The process executes iterations until the chosen number of cluster centers have been initialized.

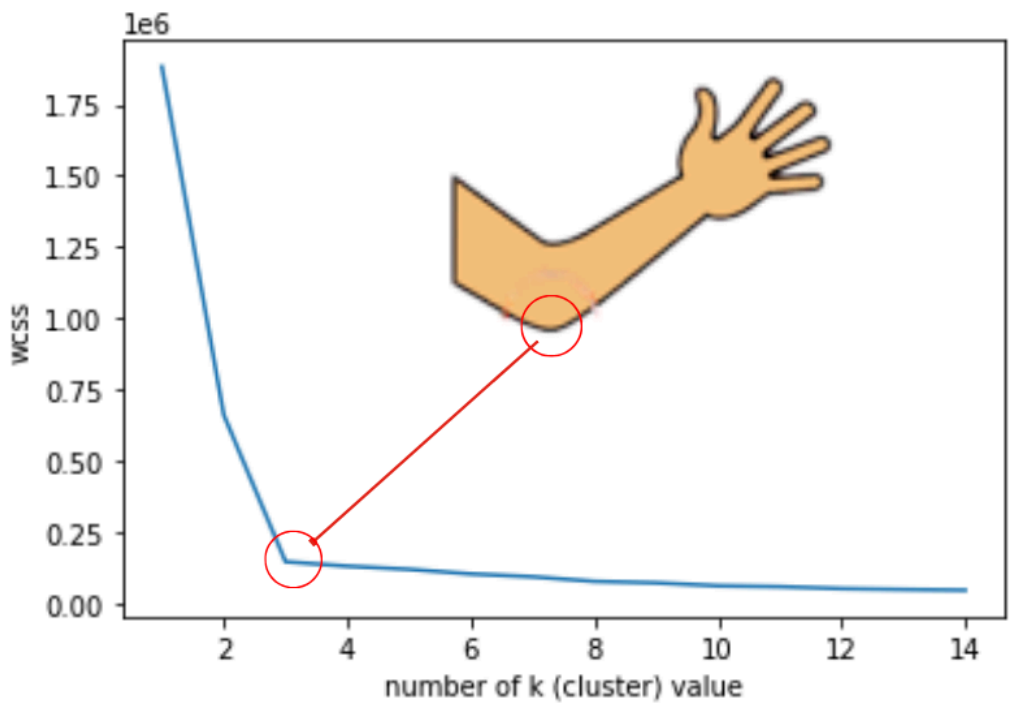


ii) Choosing the optimal number of clusters

The Elbow Method

The elbow method is a graphical method for finding the optimum number of clusters within a k-means clustering algorithm. It measures the euclidean distance between each data point and its cluster center and chooses the number of clusters based on where change in **"within cluster sum of squares" (WCSS)** levels off. This value represents the total variance within each cluster that gets plotted against the number of clusters.

The first step of the elbow method is to calculate the WCSS for each cluster (k). Then, the WCSS value is plotted along the y-axis and the number of clusters is plotted on the x-axis. As the number of clusters increases, the plot points should form a consistent pattern. From this pattern, results a range for the optimum number of clusters.8 When deciding on the number of clusters, consider the computational costs. The higher the number of clusters, the more processing power is needed especially with large datasets.



The Silhouette Analysis

In the Silhouette algorithm, we assume that the data has already been clustered into k clusters by a clustering technique(Typically K-Means Clustering technique). Then for each data point, we define the following:-

- **C(i)** -The cluster assigned to the **ith** data point.

- **|C(i)|** - The number of data points in the cluster assigned to the **ith** data point
- **a(i)** - It gives a measure of how well assigned the **ith** data point is to its cluster.

$$a(i) = \frac{1}{|C(i)| - 1} \sum_{C(i),\, i \neq j} d(i, j)$$

- **b(i) -** It is defined as the average dissimilarity to the closest cluster which is not it's cluster.

$$b(i) = min_{i \neq j} \left(\frac{1}{|c(i)|} \sum_{j \in C(j)} d(i, j)\right)$$

- The silhouette coefficient S(i) is given by:

$$s(i) = \frac{b(i) - a(i)}{max(a(i), b(i))}$$

We determine the average silhouette for each value of k and for the value of k which has the **maximum value of s(i)** is considered the optimal number of clusters for the unsupervised learning algorithm

ALGORITHM OF K-MEANS CLUSTERING

- **Step 1: Specify the number of clusters (k).** The first step in the algorithm k-means is to specify the number of clusters, which is referred to as k. Traditionally, researchers will conduct k-means multiple times, exploring different numbers of clusters (e.g., from 2 through 10).
- **Step 2: Allocate objects to clusters.** The most straightforward approach is to randomly assign objects to clusters, but there are many other approaches (e.g., using hierarchical clustering).
- **Step 3: Compute Cluster Means.** For each cluster, the average value is computed for each of the variables.
- **Step 4: Allocate each observation to the closest cluster center.**
- **Step 5: Repeat steps 3 and 4 until the solution converges (**i.e., reallocating observations and updating means cannot improve the solution**).**

CODE AND OUTPUT

In [40…

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

In [41…

```python
df = pd.read_csv('Iris.csv')
```

In [41…

```python
df.head()
```

Out[41…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [41…

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

In [41…

```python
df.describe()
```

Out[41…

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidth
count	150.000000	150.000000	150.000000	150.000000	150.0000
mean	75.500000	5.843333	3.054000	3.758667	1.1986
std	43.445368	0.828066	0.433594	1.764420	0.7631
min	1.000000	4.300000	2.000000	1.000000	0.1000
25%	38.250000	5.100000	2.800000	1.600000	0.3000
50%	75.500000	5.800000	3.000000	4.350000	1.3000
75%	112.750000	6.400000	3.300000	5.100000	1.8000
max	150.000000	7.900000	4.400000	6.900000	2.5000

In [43…

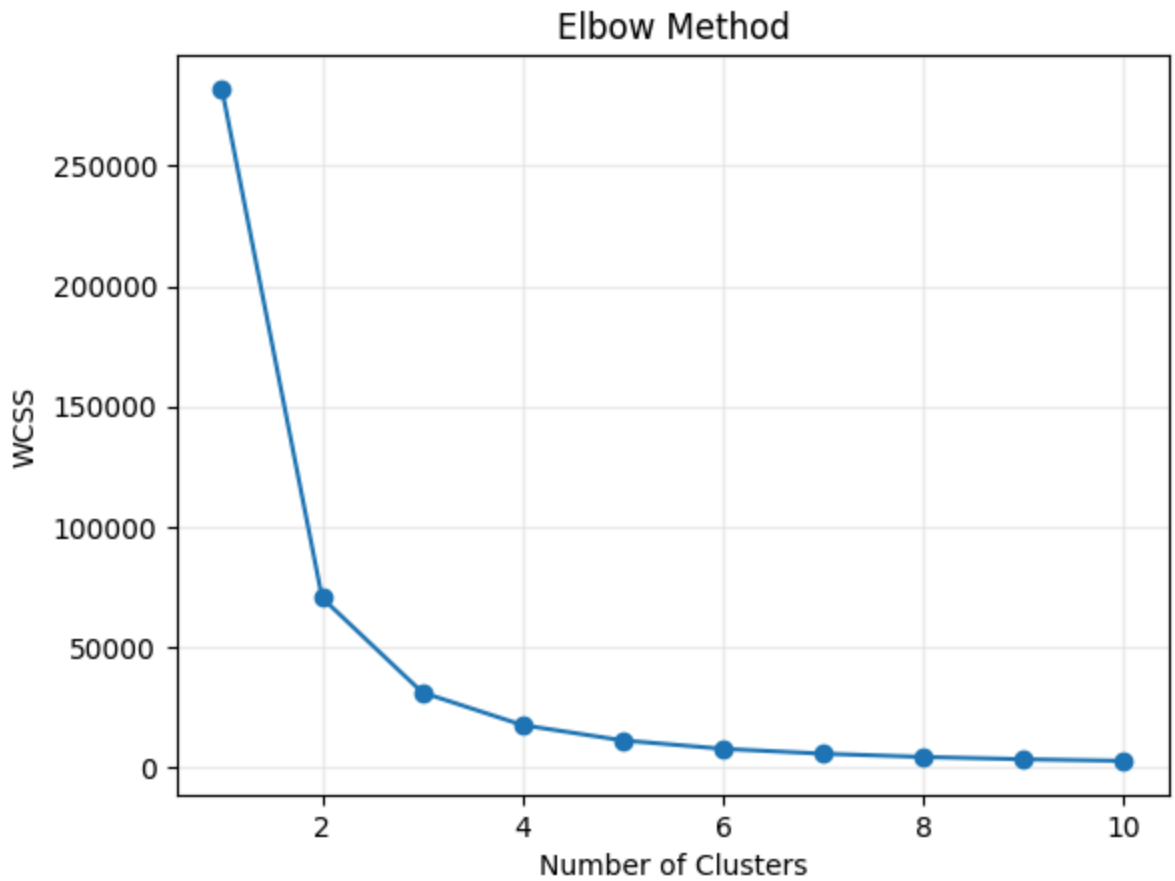
```
feature_columns = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm
x = df[feature_columns].values
```

In [41…

```
# 1) Elbow method
wcss = []
k_values = list(range(1, 11))
for i in k_values:
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300,
    kmeans.fit(x)
    wcss.append(kmeans.inertia_)
```

In [43…

```
plt.plot(k_values, wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.grid(alpha=0.2)
plt.show()
```



In [42…

```
# 2) Silhouette score on k from 2 to 10 for K-Means
silhouette_scores = {}
for i in range(2, 11):
    labels = KMeans(n_clusters=i, init='k-means++', random_state=4
    silhouette_scores[i] = silhouette_score(x, labels)

silhouette_k = max(silhouette_scores, key=silhouette_scores.get)
final_k = silhouette_k
print("Silhouette K Value: ", silhouette_k)
```

```
Silhouette K Value:  2
```

In [ ]:

```
# Mapping Cluster Id to actual Species Name
true_species = df['Species'].copy() if 'Species' in df.columns els

kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, n_ir
y_kmeans = kmeans.fit_predict(x)
df['Cluster'] = y_kmeans

if true_species is not None:
    mapping_df = pd.DataFrame({'Cluster': y_kmeans, 'Species': tru
    cluster_to_species = mapping_df.groupby('Cluster')['Species'].
else:
    center_order = np.argsort(kmeans.cluster_centers_[:, 2])
    ordered_names = ['Iris-setosa', 'Iris-versicolor', 'Iris-virgi
```

```
    cluster_to_species = {int(c): ordered_names[i] for i, c in enu

df['Species'] = pd.Series(y_kmeans).map(cluster_to_species)
print('Cluster name mapping:', cluster_to_species)
```

```
Cluster name mapping: {0: 'Iris-setosa', 1: 'Iris-versicolour', 2:
'Iris-virginica'}
```

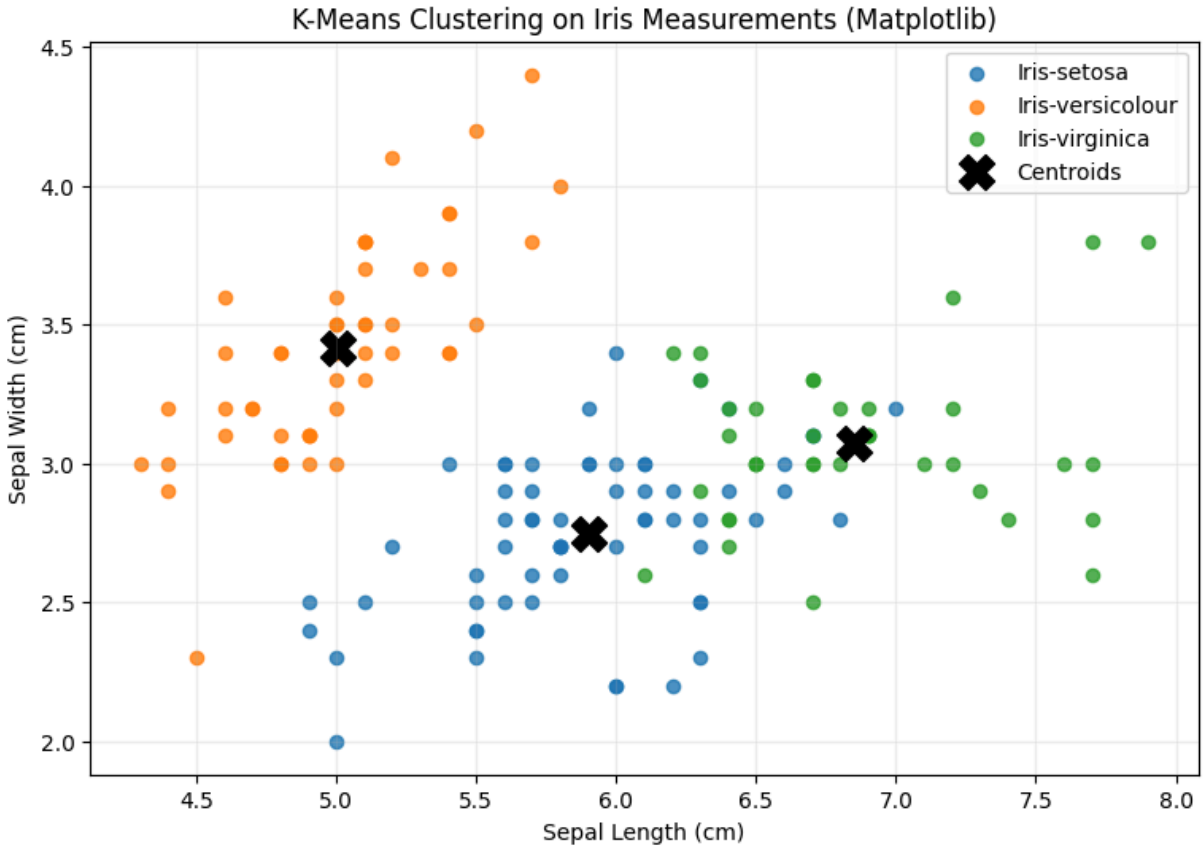
In [43…

```
feature_x_name = 'Sepal Length (cm)'
feature_y_name = 'Sepal Width (cm)'

plt.figure(figsize=(9, 6))
for cluster_id in range(kmeans.n_clusters):
    mask = y_kmeans == cluster_id
    cluster_label = cluster_to_species.get(cluster_id, f'Cluster {
    plt.scatter(
        x[mask, 0],
        x[mask, 1],
        s=35,
        alpha=0.8,
        label=cluster_label
    )

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=250,
    c='black',
    marker='X',
    label='Centroids'
 )

plt.title('K-Means Clustering on Iris Measurements (Matplotlib)')
plt.xlabel(feature_x_name)
plt.ylabel(feature_y_name)
plt.legend()
plt.grid(alpha=0.2)
plt.show()
```


K-Means Clustering on Iris Measurements (Matplotlib)
Iris-setosa
Iris-versicolour
Iris-virginica
Centroids
Sepal Width (cm)
Sepal Length (cm)
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

CONCLUSION

In this lab, we implemented the **k-means clustering** algorithm to group unlabeled data points from the Iris dataset into clusters. The algorithm iteratively assigned points to the nearest centroid and updated cluster centers until convergence was reached. We evaluated clustering quality using metrics such as **inertia** and **silhouette analysis**, which helped us determine the optimal number of clusters. The **elbow method** provided a graphical approach to selecting k, while **silhouette scores** confirmed that two clusters gave the best separation in this dataset.

Through this experiment, we observed how centroid initialization (using **k-means++**) impacts clustering performance and how different evaluation metrics guide the choice of k. We successfully mapped clusters to species groups (Iris-setosa, Iris-versicolor, and Iris-virginica), demonstrating the practical application of k-means in unsupervised learning. This lab gave us hands-on experience in coding clustering algorithms, visualizing results, and understanding the importance of distance metrics and cluster evaluation in unsupervised machine learning tasks.